

**Міністерство освіти і науки України
Львівського національного університету імені Івана Франка
Факультету електроніки та комп'ютерних технологій**

Звіт

**Про виконання лабораторної роботи №3
«Створення власного класу. Методи розв'язку рівнянь»**

**Виконала:
Студентка групи ФЕП-12
Іщенко Стефанія Павлівна**

**Перевірів:
ас. Левуш П. Н.**

Львів 2026

Мета роботи: Засвоїти структуру опису класу у мові C++, навчитись реалізувати функціонал та використовувати класи у основній програмі.

Хід роботи

1. Було створено заголовковий файл Dyhotomia_class.h. У ньому описано клас Dyhotomia_class, який містить приватні поля a, b та eps для збереження меж проміжку і точності обчислення. Також у класі оголошено функцію f(double x) для обчислення значення функції та методи setValues(), setEps() і count() для реалізації методу дихотомії.

```
C Dyhotomia_class.h > Dyhotomia_class > eps
1  #ifndef DYHOTOMIA_CLASS_H
2  #define DYHOTOMIA_CLASS_H
3
4  class Dyhotomia_class{
5      private:
6      double a, b, eps;
7
8      double f(double x);
9
10     public:
11     void setValues(double val_a, double val_b);
12     void setEps(double e);
13
14     double count();
15 };
16
17
18 #endif
```

Рис.1

2. У файлі Dyhotomia_class.cpp реалізовано функцію: $3\sin \sqrt{x} + 0.35x - 3.8$

Також реалізовано методи setValues() та setEps() для запису параметрів у поля класу. У методі count() реалізовано алгоритм дихотомії: перевірка наявності кореня на проміжку, знаходження середини відрізка та поступове звуження проміжку до досягнення заданої точності ϵ .

```
Dyhotomia_class.cpp > count()
1  #include "Dyhotomia_class.h"
2  #include <iostream>
3  #include <cmath>
4
5  using namespace std;
6
7  double Dyhotomia_class::f(double x)
8  {
9      return 3 * sin(sqrt(x)) + 0.35 * x - 3.8;
10 }
11
12 void Dyhotomia_class::setValues(double val_a, double val_b)
13 {
14     a=val_a;
15     b=val_b;
16 }
17
18 void Dyhotomia_class::setEps(double e)
19 {
20     eps = e;
21 }
22
23 double Dyhotomia_class::count()
24 {
25     if (f(a)*f(b)>=0)
26     {
27         cout << "Немає розв'язку\n";
28         return 0;
29     }
30
31     double c;
32
33     while ((b-a)>=eps)
34     {
35         c = (a+b)/2;
36
37         if (f(a)*f(c)<0)
38             b = c;
39         else
40             a = c;
41     }
42
43     return (a + b) / 2;
44 }
```

Рис.2

3. У файлі main.cpp створено екземпляр класу Dyhotomia_class. Користувач вводить значення точності ϵ , після чого задається проміжок [2;3]. Далі викликається метод count(), який знаходить корінь рівняння методом дихотомії, а результат виводиться на екран.

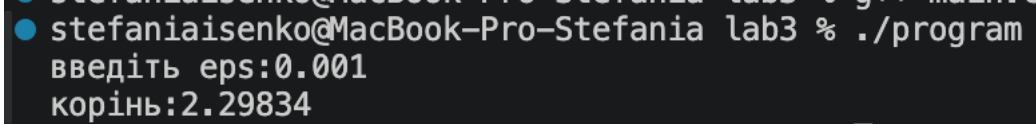
```
main.cpp > main()
1  #include <iostream>
2  #include "Dyhotomia_class.h"
3
4  using namespace std;
5
6  int main(){
7      Dyhotomia_class obj;
8      double eps;
9
10     cout << "введіть eps:";
11     cin >> eps;
12
13     obj.setValues(2, 3);
14     obj.setEps(eps);
15
16     double root=obj.count();
17
18     cout<<"корінь:"<<root<<endl;
19     return 0;
20 }
```

Рис.3

4. Програму для методу дихотомії було скомпільовано та запущено через термінал. Після введення значення ϵ програма обчислила корінь рівняння методом дихотомії. У результаті було отримано значення:

2.29834

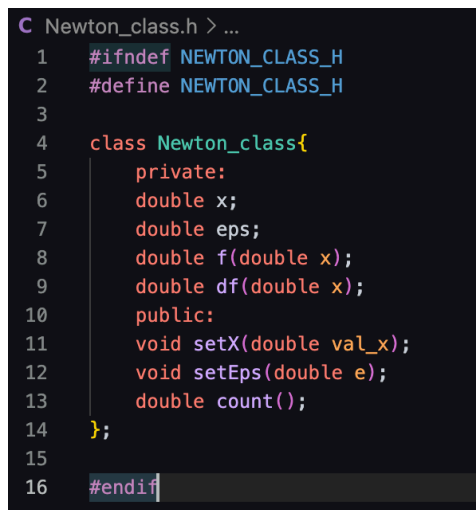
що підтверджує правильність реалізації алгоритму.



```
stefaniaisenko@MacBook-Pro-Stefania lab3 % ./program
введіть eps:0.001
корінь:2.29834
```

Рис.4

5. Було створено заголовковий файл Newton_class.h. У ньому описано клас Newton_class, який містить приватні поля x та ϵ , функцію $f(\text{double } x)$ для обчислення значення функції, функцію $df(\text{double } x)$ для обчислення похідної, а також методи $\text{setX}()$, $\text{setEps}()$ і $\text{count}()$ для реалізації методу Ньютона.



```
C Newton_class.h > ...
1  #ifndef NEWTON_CLASS_H
2  #define NEWTON_CLASS_H
3
4  class Newton_class{
5      private:
6          double x;
7          double eps;
8          double f(double x);
9          double df(double x);
10     public:
11         void setX(double val_x);
12         void setEps(double e);
13         double count();
14     };
15
16 #endif
```

Рис.5

6. У файлі `Newton_class.cpp` реалізовано функцію: $3\sin \sqrt{x} + 0.35x - 3.8$. Для чисельного обчислення похідної реалізовано метод `df(double x)` за формулою: $f'(x) = \frac{hf(x+h) - f(x)}{h}$. У методі `count()` реалізовано алгоритм Ньютона за формулою: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. Також реалізовано перевірку ділення на нуль у випадку, якщо похідна дорівнює нулю.

```
Newton_class.cpp > f(double)
1  #include "Newton_class.h"
2  #include <iostream>
3  #include <cmath>
4
5  using namespace std;
6
7  double Newton_class::f(double x)
8  {
9      return 3 * sin(sqrt(x)) + 0.35 * x - 3.8;
10 }
11 double Newton_class::df(double x){
12     double h=0.0001;
13     return (f(x+h)-f(x))/h;
14 }
15 void Newton_class::setX(double val_x){
16     x=val_x;
17 }
18 void Newton_class::setEps(double e){
19     eps=e;
20 }
21 double Newton_class::count(){
22     double x1;
23     while (fabs(f(x))>=eps){ // fabs - модуль числа
24         if(df(x)==0){
25             cout<<"Ділення на 0\n";
26             return 0;
27         }
28         x1=x-f(x)/df(x);
29         x=x1;
30     }
31     return x;
32 }
```

Рис.6

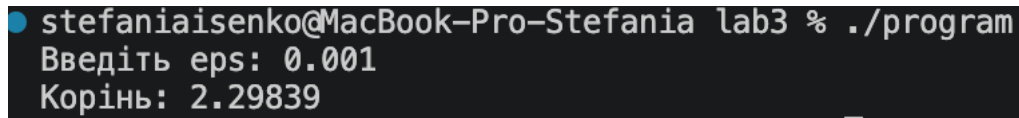
7. У файлі `main2.cpp` створено екземпляр класу `Newton_class`. Користувач вводить значення точності ϵ , задається початкове наближення $x = 2.5$, після чого викликається метод `count()` для знаходження кореня методом Ньютона. Результат виконання програми виводиться на екран.

```
main2.cpp > ...
1  #include <iostream>
2  #include "Newton_class.h"
3
4  using namespace std;
5
6  int main()
7  {
8      Newton_class obj;
9
10     double eps;
11
12     cout << "Введіть eps: ";
13     cin >> eps;
14
15     obj.setX(2.5);
16
17     obj.setEps(eps);
18
19     double root = obj.count();
20
21     cout << "Корінь: " << root << endl;
22
23     return 0;
24 }
```

Рис.7

8. Програму для методу Ньютона було скомпільовано та запущено через термінал. Після введення значення ϵ програма обчислила корінь рівняння методом Ньютона. У результаті було отримано значення:

Отриманий результат практично збігається з результатом методу дихотомії, що підтверджує правильність реалізації методу Ньютона.

A terminal window with a dark background and light-colored text. The prompt is 'stefaniaisenko@MacBook-Pro-Stefania lab3 %'. The user has entered './program'. The program outputs 'Введіть eps: 0.001' and 'Корінь: 2.29839'.

```
stefaniaisenko@MacBook-Pro-Stefania lab3 % ./program
Введіть eps: 0.001
Корінь: 2.29839
```

Рис.8

Висновок: у ході лабораторної роботи було створено власні класи для реалізації методів дихотомії та Ньютона для розв’язання нелінійних рівнянь у мові C++. Було реалізовано інкапсуляцію даних за допомогою модифікатора `private`, а також методи `public` для роботи з параметрами класів. Для заданої функції: $3\sin \sqrt{x}+0.35x-3.8$ на проміжку $[2;3]$ обома методами було отримано близькі значення кореня. У результаті виконання роботи було закріплено навички створення класів, реалізації чисельних методів та компіляції програм у середовищі C++.